itthynk

SOFTWARE DEVELOPMENT INTERNSHIP PROGRAMME · COHORT 2026

Demo moved · now Monday 14 Sep, not Friday

Appointment Booking System — Week 4 Feedback

Registration and the database schema are genuinely solid work — the validation, the hashing, the duplicate-email handling are all done properly. The pattern worth slowing down on is what happens to the session once someone's logged in: where it's kept, whether it can be ended, and how many different places re-check the same thing instead of one.

Repo · `cohort-2026/appointment-booking-system`

Sprint work · Authentication & Authorization, Mon 7 – Fri 11 Sep 2026 **Demo** · Monday 14 Sep 2026 (moved from Friday)

WHAT THIS DOCUMENT IS — AND ISN'T

This is **feedback, not a fix-it list.** Every item below is here to help you see where your own reasoning went wrong and think through a better way to approach it — not to hand you code to paste in. Each one points at the real file and line in your repo so you can go look at it yourselves, and some show the actual code that's there right now — not because copying a replacement is the point, but because seeing your own code named directly is more useful than a description of it. The short examples further down are illustrative, to make an idea concrete, not to be copied.

What to demo on Monday

The brief's own sequence, walked through against what this repo actually does today — from the same audit this feedback is drawn from.

Step 1 Register a new account

Ready

ON SCREEN

Works cleanly — the form submits and shows a success state.

IN THE CODE / TERMINAL

201 response, bcrypt hash confirmed in the DB, no `password_hash` anywhere in the JSON.

Step 2 Log in

Wrong storage

ON SCREEN

Redirects to a role-appropriate dashboard.

IN THE CODE / TERMINAL

DevTools → Application → Local Storage: the raw JWT sits there in plaintext. Application → Cookies is empty.

This is the headline problem. Any script on the page — including one from a compromised dependency — can read this token. See Solution 1.

Step 3 Refresh the page

Works, wrong way

ON SCREEN

Still appears signed in.

IN THE CODE / TERMINAL

Only because each dashboard independently re-reads `localStorage` on mount (e.g. `Dashboard.tsx:19-20`) — there's no shared `AuthContext` driving this.

Step 4 Visit a booking / provider / availability page

Not built

ON SCREEN

Nothing to show yet.

IN THE CODE / TERMINAL

`server.ts` only mounts `authRoutes` — no business routes exist to hit.

Step 5 Log out

Doesn't invalidate

ON SCREEN

Returns to the login screen.

IN THE CODE / TERMINAL

Only clears `localStorage` client-side — there's no logout endpoint, so the token stays valid if anyone captured it.

See Solution 2 — a real logout has to happen on the server.

Step 6 Check the CORS + credentials setup

Not configured

ON SCREEN

N/A — check the Network tab instead.

IN THE CODE / TERMINAL

No `Access-Control-Allow-Credentials` header on any response — `server.ts:8` is a bare `cors()` with no options.

> See Solution 3 — needed before the cookie fix in Solution 1 will actually work.

RECOMMENDED ORDER TO WORK THROUGH THIS

WHEN	ISSUE	WHERE
1st	Move the session into a cookie and fix CORS together — neither works without the other	Lessons 1 & 3
2nd	Add a real logout endpoint	Lesson 2
3rd	Wire up the shared AuthContext/ProtectedRoute instead of four separate checks	Lesson 4
4th	Add the missing email validation, and wire the middleware into real business routes as they're built	Lessons 7 & 8
anytime	Update or delete the stale schema.sql, and add a seed script for demo accounts	Lessons 5 & 6

The session/CORS fix unblocks everything else that depends on being signed in — without it, nothing downstream can even be tested properly.

Where this went wrong, and how to think about it differently

In the order it'll matter most — not necessarily the order you'll want to tackle it.

1 Deciding where the session lives, and why that choice matters

`frontend/src/pages/public/Login.tsx:40`

WHAT WENT WRONG

> After login, the session token is stored in the browser's localStorage. It works — the app does stay signed in — but localStorage is readable by any JavaScript running on the page, which includes code from any library the project depends on.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
localStorage.setItem("token", data.token);
```

WHY IT MATTERS

> *Think of the difference between a key kept in a locked safe that only the front door can open automatically, versus a key left under the doormat where it's convenient — for you, and for anyone else who thinks to look there. Both let you in. Only one is actually protecting anything.*

A BETTER WAY TO THINK ABOUT IT

> Let the server hand the session to the browser as a cookie marked httpOnly, instead of in the response body for the frontend to store itself. The browser then attaches it automatically on future requests, and no JavaScript on the page — yours or a dependency's — can ever read it directly.

EXAMPLE

illustrative only

```
// instead of the frontend doing this:
localStorage.setItem("token", data.token);

// the backend does this once, and the frontend never touches the token at all:
res.cookie("session", token, { httpOnly: true, sameSite: "lax" });
```

2

A door that locks, but was never given a way to be unlocked from the other side

`backend/src/routes/auth.routes.ts · backend/src/controllers/auth.controller.ts`

WHAT WENT WRONG

> There's no logout endpoint. The button on the screen only clears the browser's local copy of the token — but the token itself is still valid for the rest of its lifetime, up to a full day, no matter what the UI shows.

WHAT'S ACTUALLY THERE RIGHT NOW

Grepping the whole backend for `logout` turns up nothing — no route, no controller function. "Logout" only exists on the frontend, as a `localStorage` clear.

WHY IT MATTERS

> *It's like deleting a photo of your hotel key card from your phone and assuming the door no longer opens for it. The card still works. You've only removed your own reminder that you have it.*

A BETTER WAY TO THINK ABOUT IT

> Logging out needs to be something the server does, not just the browser. The server is the only thing that can actually say "this session is over" — clearing the cookie from its side is what makes the session genuinely stop working, not just stop being visible.

EXAMPLE

illustrative only

```
app.post("/api/auth/logout", (req, res) => {
  res.clearCookie("session");
  res.json({ message: "Logged out" });
});
// this is what actually ends the session — not the frontend forgetting it
```

3 Two settings that only do anything as a pair `backend/src/server.ts:8`

WHAT WENT WRONG

> The server currently accepts requests from any origin with no special handling for cookies. On its own that looks harmless — but it means the two settings that make cross-origin cookies work at all (one on the server, one on the frontend) are both missing, and neither half being present quietly does nothing.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
app.use(cors());
```

WHY IT MATTERS

> *It's like a phone call where you've agreed to record it, but the other person hasn't consented — the recording doesn't happen at all, for either of you, even though you did your part. Both sides have to opt in for it to work; one side alone accomplishes nothing.*

A BETTER WAY TO THINK ABOUT IT

> Treat the CORS setting on the server and the credentials setting on the frontend as one decision, not two separate ones — change them together, and test the pair together, rather than assuming that getting one right is progress on its own.

EXAMPLE

illustrative only

```
// server:
app.use(cors({ origin: "http://localhost:5173", credentials: true }));

// frontend — every request, not just some of them:
fetch(url, { credentials: "include" });
// neither line does anything useful without the other
```

4 The same check, written four separate times instead of once

`frontend/src/context/AuthContext.tsx, hooks/useAuth.ts, routes/ProtectedRoute.tsx, routes/RoleRoute.tsx`

WHAT WENT WRONG

> Right now, each dashboard page independently reads the browser's storage and decides for itself whether someone is signed in. There's a shared place meant to hold this (an AuthContext file) — but it was left empty, so every page re-invented the same logic on its own.

WHAT'S ACTUALLY THERE RIGHT NOW

All four of these files are confirmed empty (0 bytes). Each dashboard page — customer, provider, business — has its own separate `localStorage` check and its own `useEffect` redirect instead.

WHY IT MATTERS

> *Imagine four different staff members at a building each keeping their own separate visitor logbook instead of sharing one at the front desk. It mostly works, until someone forgets to update their copy — and now the four logbooks disagree with each other about who's actually allowed inside.*

A BETTER WAY TO THINK ABOUT IT

> Build the shared logic once, in the place already set aside for it, and have every page read from that single source instead of repeating the check. The day the rule for "signed in" changes, there's exactly one place to update it — not four places to remember.

EXAMPLE

illustrative only

```
// one shared place, read from everywhere:
const { user } = useAuth();

// instead of each page doing its own version of:
const user = JSON.parse(localStorage.getItem("user"));
```

5 The written plan and the real database don't agree with each other

`database/schema.sql vs backend/prisma/schema.prisma`

WHAT WENT WRONG

> There's a schema.sql file in the project describing tables like businesses and provider_profiles — but the database actually used by the app (defined in Prisma) has a completely different shape, with a role column and no businesses table at all. One of these documents is describing a system that doesn't exist anymore.

WHAT'S ACTUALLY THERE RIGHT NOW

`schema.sql` describes `businesses`, `business_memberships`, and `provider_profiles` tables with a `full_name` column and no `role` column. The Prisma schema the app actually uses has none of that — a different, current design.

WHY IT MATTERS

> *It's like a building still displaying its original architectural blueprints in the lobby, years after a renovation changed the layout completely — anyone using the blueprint to find a room ends up somewhere that used to be there and isn't anymore.*

A BETTER WAY TO THINK ABOUT IT

> When a design changes, the old description needs to either be updated to match or removed. An outdated plan sitting next to the real one is worse than no plan at all, because it looks authoritative right up until someone trusts it.

EXAMPLE

illustrative only — the shape of the mismatch, not literal excerpts

```
-- database/schema.sql describes tables like:
CREATE TABLE businesses (...);
CREATE TABLE provider_profiles (full_name TEXT, ...);

// but backend/prisma/schema.prisma — what the app actually queries — has:
model User { id String @id, email String @unique, role Role, ... }
// no businesses table, no full_name column — two different designs
```

6 Nobody has an account yet, except whoever registers one live

`backend/src/seed.ts`

WHAT WENT WRONG

> There's no seed script that creates any user accounts — only sample "Package" data. Anyone wanting to log in and try the app has to register a brand new account first, every time the database is reset.

WHAT'S ACTUALLY THERE RIGHT NOW

`seed.ts` only seeds `Package` rows — there is no user-seeding logic anywhere in the repo.

WHY IT MATTERS

> *It's like opening a shop with fully stocked shelves but no staff badges made yet — the shop looks ready, but nobody can actually clock in without someone printing a badge for them first.*

A BETTER WAY TO THINK ABOUT IT

> A small seed script that creates one account per role costs a few minutes to write and saves everyone — teammates, reviewers, you right before a demo — from re-registering by hand every time the database resets.

EXAMPLE

illustrative only

```
// backend/prisma/seed.ts
const roles = ["customer", "provider", "business"];
for (const role of roles) {
  await prisma.user.create({
    data: { email: `${role}@demo.test`, passwordHash: await
bcrypt.hash("Demo1234!", 12), role },
  });
}
```

7

The email field only checks that something was typed, not that it looks like an email

`backend/src/controllers/auth.controller.ts`

WHAT WENT WRONG

> Server-side, the email field is only checked for being non-empty — there's no check that it's actually shaped like an email address. The only thing stopping "not-an-email" from being accepted is the frontend's input type, which anyone bypassing the form skips entirely.

WHAT'S ACTUALLY THERE RIGHT NOW

Every register variant checks `!email` (is it empty) but none of them check the email's actual format — that check exists only on the frontend's `<input type="email">`.

WHY IT MATTERS

> *It's similar to a form that only checks a box was filled in, not what's written in it — "yes", "banana", and a real answer are all treated the same as long as something's there.*

A BETTER WAY TO THINK ABOUT IT

> Server-side validation should assume the frontend doesn't exist — because for a fair number of legitimate cases (an API client, a script, a bypassed form) it effectively doesn't. If the rule matters, it has to be checked again where the data is actually used.

EXAMPLE

illustrative only

```
if (!email) return res.status(400).json({ message: "Email is required" });
// add the part that's currently missing:
if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
  return res.status(400).json({ message: "Invalid email format" });
}
```

8

The permission check exists, but almost nothing has been asked to use it yet

`backend/src/server.ts · only authRoutes is mounted`

WHAT WENT WRONG

> The authentication and role-checking middleware are built well — but only one placeholder demo route actually calls them. None of the real business routes are wired into the server yet, so there's currently nothing real for the permission check to actually be protecting.

WHAT'S ACTUALLY THERE RIGHT NOW

`authenticate`/`requireRole` are only used by one placeholder route, `GET /api/auth/customer-test`. No booking, provider, or availability routes are registered in `server.ts` at all yet.

WHY IT MATTERS

> *It's like installing a well-built lock on a door frame before the door itself has been hung — the lock mechanism is sound, but there's nothing yet for it to be locking.*

A BETTER WAY TO THINK ABOUT IT

> It's a fine order to build in — mechanism first, then attach it everywhere it's needed — as long as attaching it to every real route is tracked as the next concrete step, not assumed to follow automatically once the mechanism exists.

EXAMPLE

illustrative only

```
// currently only this exists:
router.get("/customer-test", authenticate, requireRole("customer"), handler);

// the real routes still need the same treatment once they're built:
router.get("/bookings", authenticate, requireRole("customer"), listBookings);
router.post("/bookings", authenticate, requireRole("customer"), createBooking);
```

Checklist — work through this before Monday

One box per lesson above, phrased as something you can actually check off. Tick it once you can point to where it's true in your own code — not once you've read the explanation. Items marked **Critical** are where the audit found a real gap; be ready to show it live, not describe it.

- [ ] **Is the session an httpOnly cookie the browser holds, instead of something stored in localStorage?** CRITICAL
- [ ] **Does calling logout actually invalidate the session server-side, not just clear the client?** CRITICAL
- [ ] **Is CORS set to a specific origin with credentials:true, matched by credentials:'include' on every fetch?** CRITICAL
- [ ] **Do all four dashboard pages read from one shared AuthContext instead of their own localStorage check?** CRITICAL
- [ ] Does database/schema.sql actually match the tables Prisma is using, or has one gone stale?
- [ ] Is there a seed script that creates at least one demo account per role?
- [ ] Does the server reject a malformed email address, not just an empty one?

- [ ] Are the real booking/provider/availability routes wired into the server and protected by the existing middleware?
